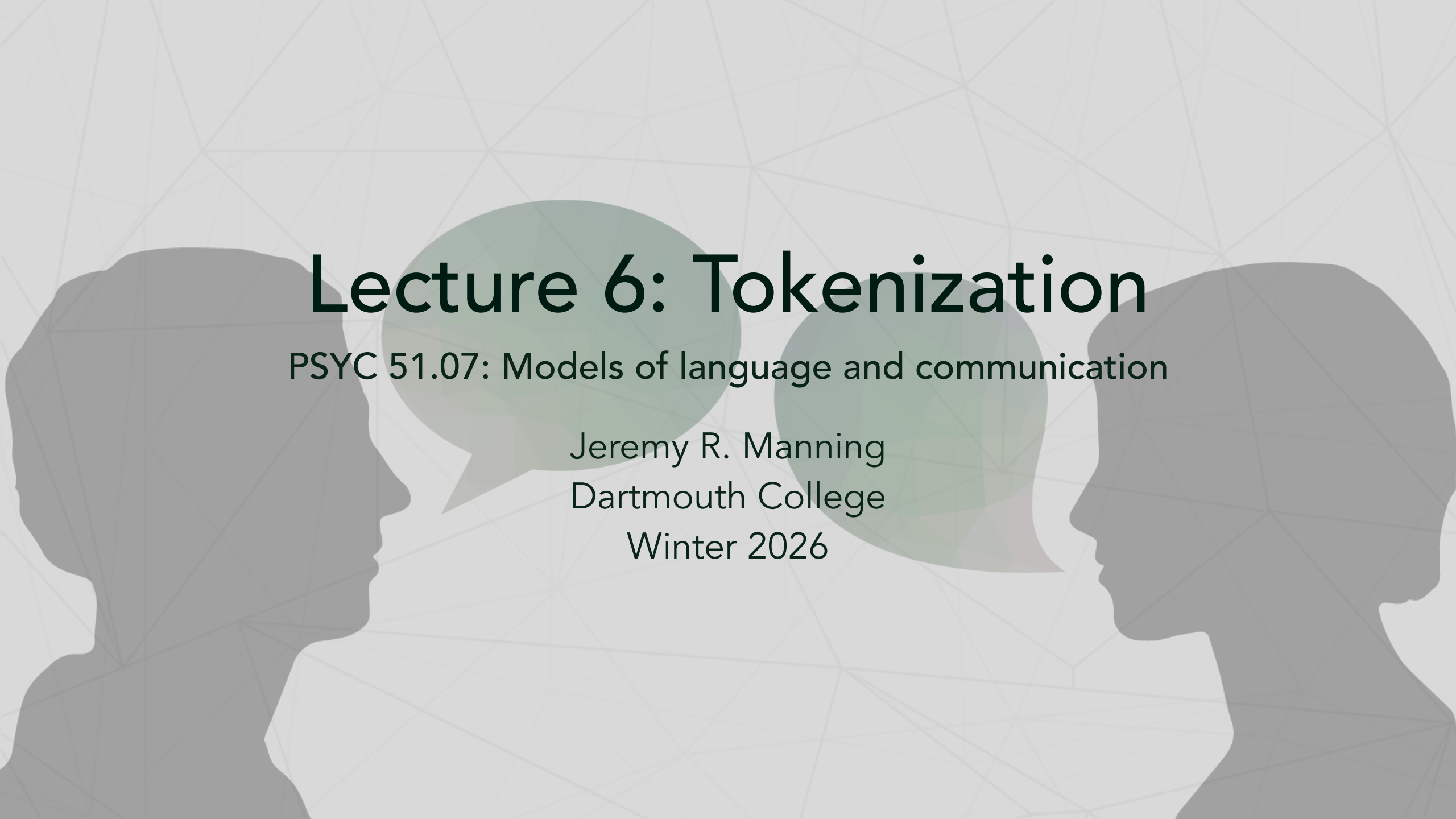


Lecture 6: Tokenization

PSYC 51.07: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Understand what tokenization is and why it matters
2. Explain the limitations of word-level tokenization
3. Describe how subword tokenization works (BPE, WordPiece, SentencePiece)
4. Compare different tokenization methods and their trade-offs
5. Implement and experiment with different tokenizers using HuggingFace

Central question

How do we break text into meaningful units (for analyzing text, AI models, etc.)?

What is tokenization?

Definition

Tokenization is the process of converting text into smaller units (tokens).

Key idea

The granularity of tokenization determines *what a model can learn*

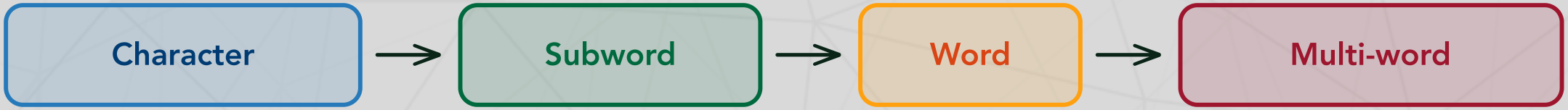
Why tokenize?

- Neural networks process numbers, not text
- Need discrete units to create vocabulary
- Represent text efficiently (compression!)
- Enable learning patterns at different levels
- First step in (nearly) any NLP pipeline

What can tokens be?

- **Bytes:** 0-255 (most fine-grained)
- **Characters:** a, b, c, ... (very fine-grained)
- **Subwords:** "un", "happiness" (sweet spot!)
- **Words:** "hello", "world" (intuitive but limited)

The tokenization spectrum



Fine-grained to coarse-grained tokenization

Granularity	Vocabulary size	Sequence length
Character	~100s	Very long
Subword	30k-50k	Medium
Word	100k+	Short
Multi-word	Millions	Very short (rarely used in practice)

Splitting on spaces fails for three reasons

1. Vocabulary explosion

English has ~170,000 commonly used words, where each inflection counts separately ("run", "runs", "running", "ran"), and compounds vary ("ice cream", "icecream", "ice-cream")

2. Unknown words

New terms ("COVID-19", "selfie"), rare words ("supercalifragilisticexpialidocious"), and typos ("teh") may be missing from training data, leading to out-of-vocabulary (OOV) issues.

3. Languages without spaces

Chinese: 没有空格 | Japanese: 日本語も同様 | Thai: **เหมือนกัน** | Emojis: 😊🚀

Word-level limitations illustrated

Vocabulary growth as training data increases

Observation

Word-level vocabulary keeps growing! Subword vocabulary stabilizes at a reasonable size.

The OOV problem in action

```python
# Simple word-level vocabulary
vocab = {"hello", "world", "the", "cat", "sat"}

def tokenize_word_level(text, vocab):
 tokens = text.lower().split()
 result = []
 for token in tokens:
   if token in vocab:
     result.append(token)
   else:
     result.append("<UNK>") # Unknown token
 return result

# Example
text = "Hello! The cat jumped"
tokens = tokenize_word_level(text, vocab)
print(tokens)
# Output: ['hello', 'the', 'cat', '<UNK>']
# ^^^ "jumped" is unknown!
```

Subword tokenization to the rescue: most words are made of smaller meaningful pieces

Examples:

- "unhappiness" = "un" + "happiness"
- "preprocessing" = "pre" + "process" + "ing"
- "antiestablishment" = "anti" + "establish" + "ment"
- "running" = "run" + "ning"
- "cats" = "cat" + "s"

Benefits:

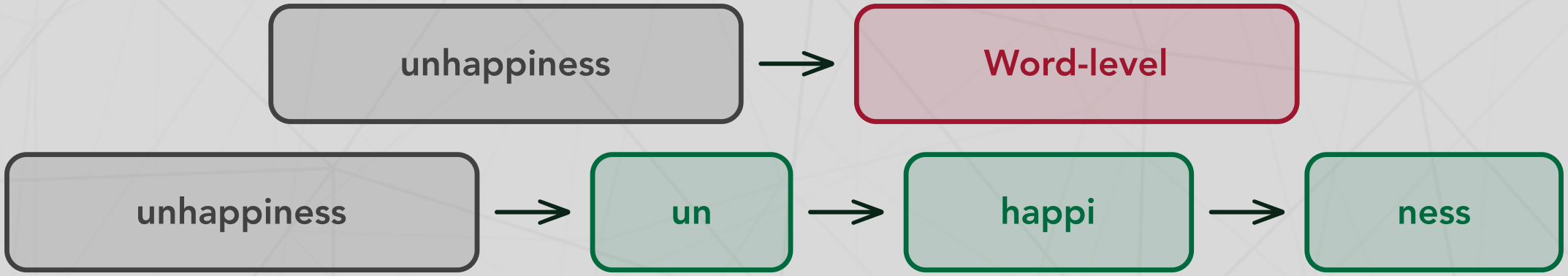
- Fixed vocabulary (30k-50k tokens)
- No OOV: break into known parts (at the character level if needed)
- Captures morphology (prefixes, suffixes)
- Language-agnostic (works for many languages)

> **Key insight**
>
> Learn common character sequences from data! Works for any language.

Subword tokenization solves the OOV problem

```python
# Word-level vocabulary (only words seen in training)
vocab = {"the", "cat", "sat", "on", "mat", "dog", "ran"}

# Trying to tokenize a new sentence:
sentence = "The supercalifragilisticexpialidocious cat meowed"
# Word-level result loses almost everything: ["<UNK>", "<UNK>", "cat", "<UNK>"]

# The subword solution
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("gpt2")

# Same difficult sentence:
tokens = tokenizer.tokenize("supercalifragilisticexpialidocious")
print(tokens)
# ['super', 'cal', 'if', 'rag', 'il', 'istic', 'exp', 'ial', 'id', 'ocious']

# Every word can be tokenized! No information loss.
# Model can learn that 'super-' often means "very/above"
```

Visual: How "unhappiness" gets tokenized



Word-level (1 token, may be OOV) vs. Subword BPE (3 tokens, always known)

Trade-off

More tokens = longer sequences, but smaller vocabulary and no OOV!

Byte-Pair Encoding (BPE)

Further reading

Sennrich, Haddow, & Birch (2016, *ACL*): Neural machine translation of rare words with subword units

- **Original use:** Data compression (1994)
- **Adapted for NLP:** Sennrich et al. (2016)
- **Used by:** GPT, GPT-2, GPT-3, RoBERTa, BART, many others!

Core idea

Iteratively merge the most frequent pair of characters/subwords.

Algorithm

1. Start with character-level vocabulary
2. Count all adjacent pairs in corpus
3. Merge most frequent pair → create new token
4. Repeat until desired vocabulary size

BPE example: Step by step

Training data: "low low low lower lower newest newest newest newest widest"

Initial tokens (characters): `l, o, w, e, r, n, s, t, i, d`

Iteration 1

Most frequent pair = `e, s` (appears 5 times)

- Merge → new token: `es`
- Vocabulary: `l, o, w, e, r, n, s, t, i, d, es`

Iteration 2

Most frequent = `o, w` (appears 5 times)

- Merge → new token: `ow`
- Vocabulary: `l, o, w, e, r, n, s, t, i, d, es, ow`

Iteration 3

Most frequent = `_, n` (space + n; appears 4 times)

- Merge → new token: `_n`
- Vocabulary: `l, o, w, e, r, n, s, t, i, d, es, ow, _n`

...continue until we hit the target vocabulary size (e.g., 30,000) or run out of merges.

BPE in practice with HuggingFace

```python
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("gpt2")  # GPT-2 uses BPE

text = "I'm learning about tokenization!"
tokens = tokenizer.tokenize(text)
print("Tokens:", tokens)
# ['I', "'m", 'Ġlearning', 'Ġabout', 'Ġtoken', 'ization', '!']
# Note: 'Ġ' represents space

token_ids = tokenizer.encode(text)  # Get token IDs
print("Token IDs:", token_ids)  # [40, 1101, 4673, 546, 11241, 1634, 0]

decoded = tokenizer.decode(token_ids)  # Decode back
print("Decoded:", decoded)  # "I'm learning about tokenization!"
```

Try it out!

Use the Tokenization Explorer Demo to experiment with different tokenizers interactively.

WordPiece merges "surprisingly common" pairs

Further reading

Wu et al. (2016, *arXiv*): Google's Neural Machine Translation System

Key difference from BPE

Instead of merging most *frequent* pair, merge pair that maximizes *likelihood*:

$$\text{score}(x, y) = \frac{P(xy)}{P(x)P(y)}$$

- **Used by:** BERT, DistilBERT, Electra
- **Special tokens:** `##` prefix for continuation

"playing" → `["play", "##ing"]`

WordPiece example with BERT

```python
from transformers import AutoTokenizer

# BERT uses WordPiece
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = "I'm learning about tokenization!"

tokens = tokenizer.tokenize(text)
print("Tokens:", tokens)
# Output: ['i', "'", 'm', 'learning', 'about', 'token', '##ization', '!']
# ^^^ Note the ##. Also: BERT lowercases by default (unless using cased model)

# Compare with longer word
text2 = "unhappiness"
tokens2 = tokenizer.tokenize(text2)
print("Tokens:", tokens2)
# Output: ['un', '##hap', '##pin', '##ess']
# Breaks into morphological components!
```

SentencePiece works for languages without spaces

Further reading

Kudo & Richardson (2018, *EMNLP*): SentencePiece: a simple and language independent subword tokenizer

Problems with BPE/WordPiece

- Assumes pre-tokenized text (spaces between words)
- Fails on Chinese, Japanese, Thai...

SentencePiece solution

- Raw character stream input
- Spaces are just "another character" (`_`)

Used by: T5, ALBERT, XLNet, mT5 (multilingual models)

SentencePiece in action

```python
from transformers import AutoTokenizer

# T5 uses SentencePiece
tokenizer = AutoTokenizer.from_pretrained("t5-small")

text = "I'm learning about tokenization!"

tokens = tokenizer.tokenize(text)
print("Tokens:", tokens)
# Output: ['I', "'", 'm', 'learning', 'about', 'token', 'ization', '!']
# ^^^ Note the for spaces

# Works seamlessly with other languages!
text_chinese = "这是中文"
tokens_cn = tokenizer.tokenize(text_chinese)
print("Chinese:", tokens_cn)
# Output: ['这', '是', '中', '文']
# Breaks into characters/subwords without needing pre-tokenization
```

Tokenization methods comparison

Method	Approach	Used By	Key Feature
BPE	Frequency merging	GPT-2, RoBERTa	Bottom-up
WordPiece	Likelihood merging	BERT, DistilBERT	Principled scoring
SentencePiece (Unigram)	Top-down pruning	T5, mT5	Multilingual
SentencePiece (BPE)	Bottom-up	mBART	Works with raw text

HuggingFace Resources

- Chapter 2.4: Behind the Pipeline - Tokenizers
- Chapter 6: Tokenizers (full chapter)

Try it out!

Use the Tokenization Explorer Demo to compare tokenizers interactively:

- Explore how different tokenizers split the same text
- Examine trade-offs between how different methods handle diffrent types of text (e.g., how efficiently they tokenize different passages)
- Look at each tokenizer's vocabulary and special tokens

Comparing tokenizers side-by-side (in Python)

```python
from transformers import AutoTokenizer
text = "The unhappiest researchers couldn't preprocess data!"

models = {"GPT-2 (BPE)": "gpt2", "BERT (WordPiece)": "bert-base-uncased", "T5 (SentencePiece)": "t5-small"}

for name, model_name in models.items():
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    tokens = tokenizer.tokenize(text)
    print(f"{name}: Tokens ({len(tokens)}): {tokens}")

# Observe: Different handling of "unhappiest", spaces, and token counts!
```

Tokenizers add special tokens for model-specific purposes

Token	Purpose	Used By
[CLS]	Classification token (start)	BERT
[SEP]	Separator between segments	BERT
[PAD]	Padding to same length	Most models
[UNK]	Unknown/rare tokens	Most models
[MASK]	Masked token for training	BERT
<s>, </s>	Start/end of sequence	GPT-2, T5
<|endoftext|>	Document boundary	GPT-2

Example usage

```
[CLS] The cat sat [SEP] The dog ran [SEP]
```

Working with special tokens

```python
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = "Hello world"
encoded = tokenizer(text, return_tensors="pt")
print("Input IDs:", encoded['input_ids'])  # [101, 7592, 2088, 102]
# ^^^ [CLS] and [SEP] added automatically!

full_decode = tokenizer.decode(encoded['input_ids'][0])
print("With special:", full_decode)  # "[CLS] hello world [SEP]"

clean_decode = tokenizer.decode(encoded['input_ids'][0], skip_special_tokens=True)
print("Without special:", clean_decode)  # "hello world"
```

Tokenization pitfalls and gotchas

1. **Tokenizer-model mismatch:** Always use the tokenizer that matches your model! GPT-2 tokenizer ≠ BERT tokenizer
2. **Maximum sequence length:** BERT: 512 tokens | GPT-2: 1024 | GPT-3: 2048 — Text gets truncated if too long!
3. **Case sensitivity:** `bert-base-uncased` lowercases everything; `bert-base-cased` preserves case
4. **Rare words → many tokens:** "antidisestablishmentarianism" → 10+ tokens; can hit sequence limit faster than expected!
5. **Special characters:** Emoji, Unicode, accents may be split unexpectedly

Connection to human language learning

Further reading

Saffran, Aslin, & Newport (1996, *Science*): Statistical learning by 8-month-old infants

Infant learning:

- Babies track statistical regularities in speech
- Identify word boundaries from transitional probabilities
- No built-in rules, just patterns learned through experience!
- Online, incremental, multimodal

BPE subword tokenization:

- Merge frequent character pairs, discover subwords
- Tracks which sequences often co-occur in training data
- No built-in rules, just patterns learned from data!
- Offline batch processing, unimodal (text only)

Discussion questions

Think about it...

1. **Linguistics vs. Statistics:** BPE discovers morphemes (un-, -ing, -ness) without linguistic rules. Is this "learning" morphology, or just pattern matching?
2. **Cross-lingual tokenization:** should we use the same tokenizer for all languages? What are the trade-offs?
3. **Semantic preservation:** does breaking "unhappy" into ["un", "happy"] preserve meaning? What about "butterfly"?
4. **Human vs. machine:** humans don't consciously tokenize words. Why do machines need to?
5. **Future directions:** will we move toward character-level or byte-level models that don't need tokenization?

Key takeaways

1. **Tokenization is fundamental:** turns raw text into model-ready input
2. **Word-level has major limitations:** OOV problem, vocabulary explosion, language-dependent
3. **Subword tokenization is the sweet spot:** Balanced vocabulary size and sequence length
4. **Different methods, similar principles:** BPE: frequency-based | WordPiece: likelihood | SentencePiece: language-agnostic
5. **Statistical learning connects humans and machines:** both discover structure from distributional patterns
6. **Always match tokenizer to model!** critical for correct predictions

Looking ahead

Next lecture: X-Hour Text Classification Workshop

How tokenization connects:

- POS tagging: Token-level classification
- Sentiment: Sequence-level classification
- Both depend on good tokenization!

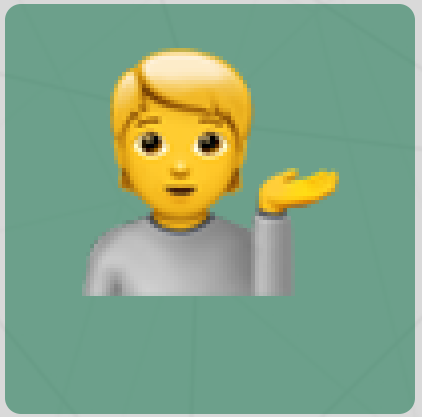
Prepare by...

- Playing more with the Tokenization Explorer Demo
- Experimenting with HuggingFace tokenizers

Questions? Want to chat more?

Email me

Join our Discord



Come to office hours

Pro tip

Remember that the ELIZA assignment is due on **Friday at 11:59 PM** — reach out if you have any questions or need help!